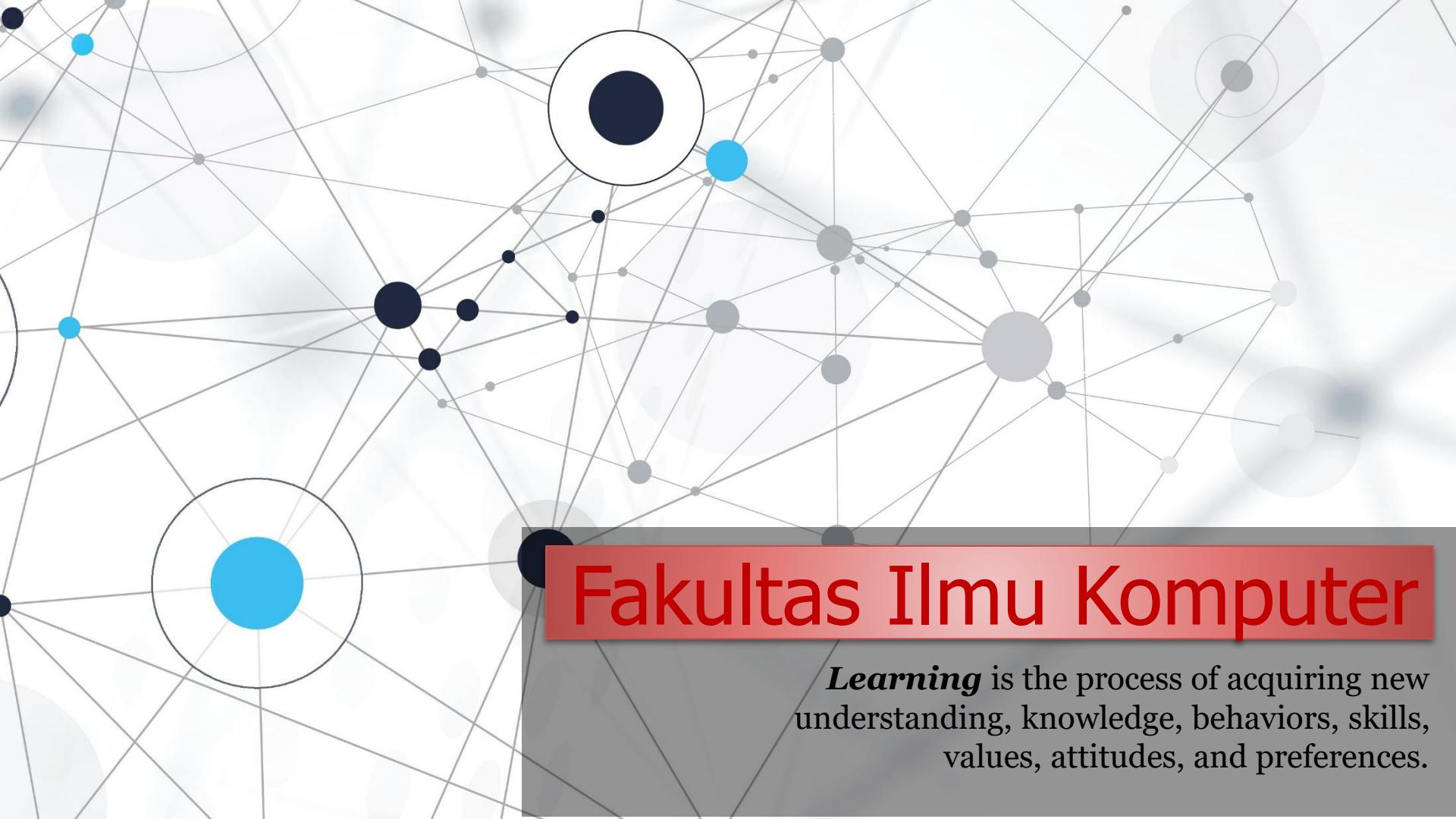


An abstract network diagram with various sized nodes (black, blue, and grey) connected by thin grey lines. Some nodes are highlighted with larger circles. The background is white with faint grey circles.

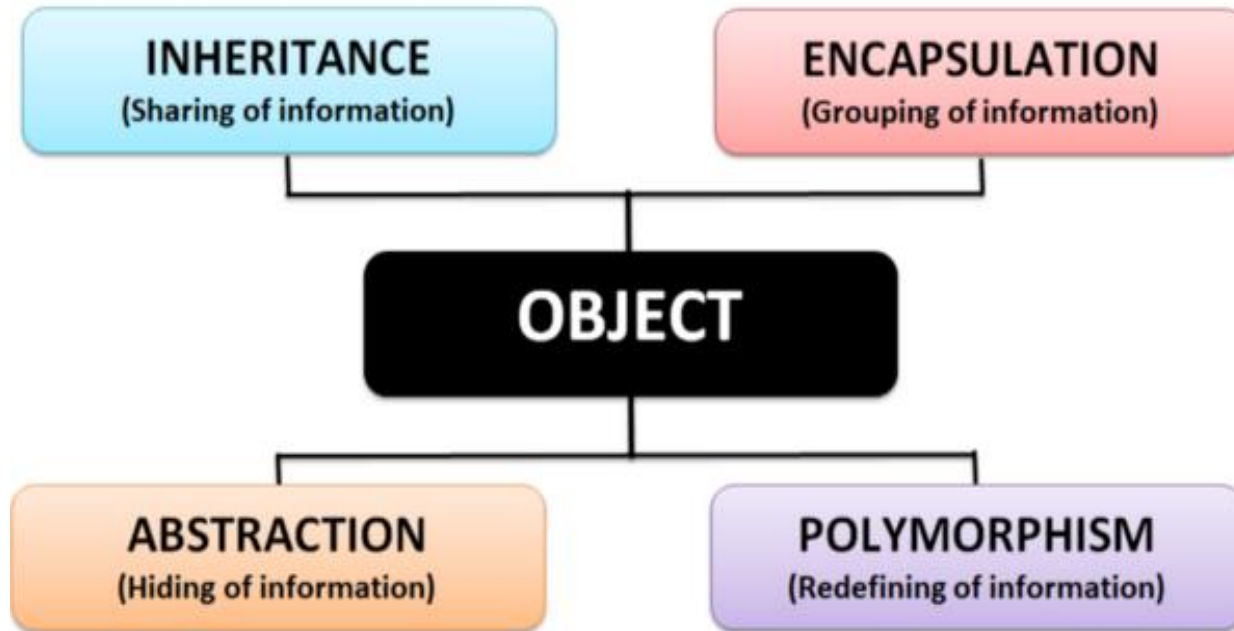
Fakultas Ilmu Komputer

Learning is the process of acquiring new understanding, knowledge, behaviors, skills, values, attitudes, and preferences.

OOP in Python – Abstraction (Data Abstraction)

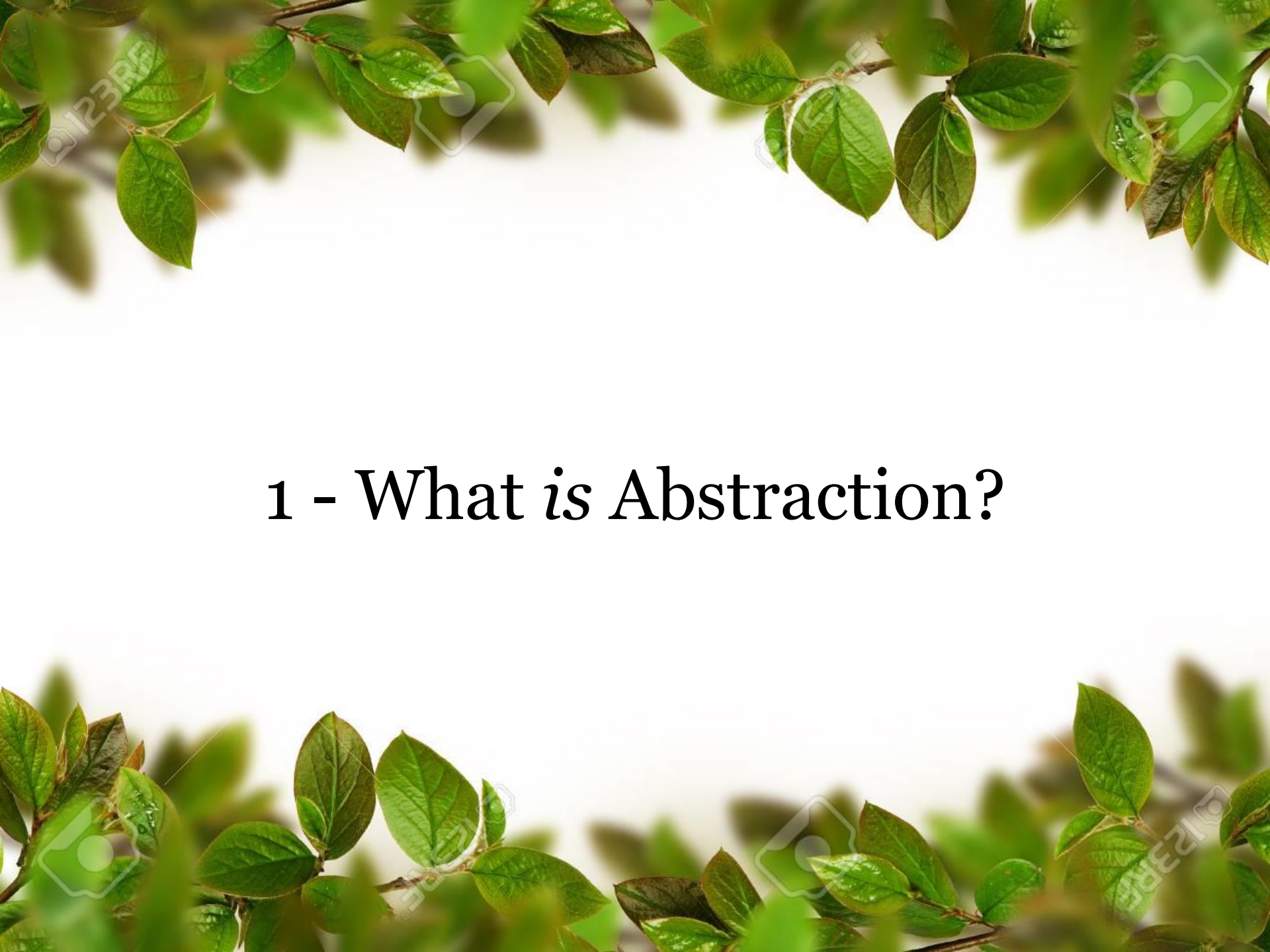


Understanding Concept of OOP



- ✓ Object Oriented Programming (OOP) atau pemrograman berorientasi objek adalah salah satu pendekatan paling efektif dalam menulis code suatu program/software.
- ✓ Pada OOP, kita menulis class yang merepresentasikan suatu situasi seperti pada keadaan sebenarnya dan kita dapat membuat suatu *object* berdasarkan class tersebut.

The four pillars of object-oriented programming in Python are: Abstraction; Encapsulation; Inheritance; Polymorphism.

The slide features a white background with green leaves and branches framing the top and bottom edges. The leaves are vibrant green and appear to be from a tree or shrub. The text is centered in the middle of the slide.

1 - What is Abstraction?

What is Abstraction?

Ilustrasi *abstraksi* menggunakan “*washing machine*”.



- ❖ When you use a washing machine for laundry purposes. What mechanism does it use?
- ❖ **A user is not required to know the engineering behind its work.**
- ❖ This process is typically known as *data abstraction*, **when all the unnecessary information is kept hidden from the users.**

What is Abstraction?

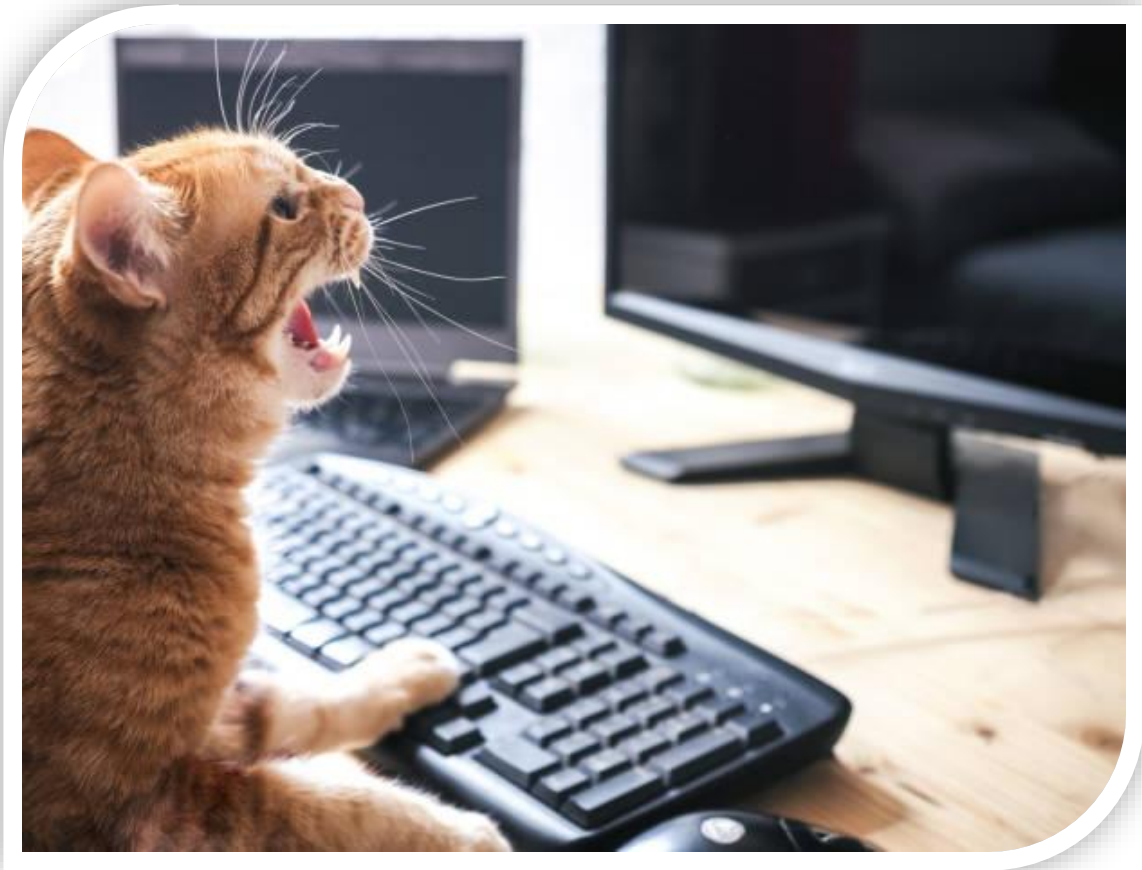
Ilustrasi *abstraksi* menggunakan “*think about a tesla car*”.



- ❖ People do not think of a car as a set of thousands of individual parts.
- ❖ Instead they see it as a well-defined object with its own unique behavior.
- ❖ **Abstraksi memungkinkan *users* untuk menggunakan mobil canggih ini untuk dikendarai tanpa mengetahui kerumitan bagian-bagian yang membentuk mobil tersebut.**
- ❖ ***Users* dapat mengabaikan detail cara kerja transmisi mesin, dan sistem pengereman. Sebaliknya, *users* bebas memanfaatkan objek secara keseluruhan.**

What is Abstraction?

Ilustrasi *abstraksi* menggunakan “computer”.



Apakah kalian
sebagai *users*,
pernah
memperhatikan
seberapa complex-
nya computer?



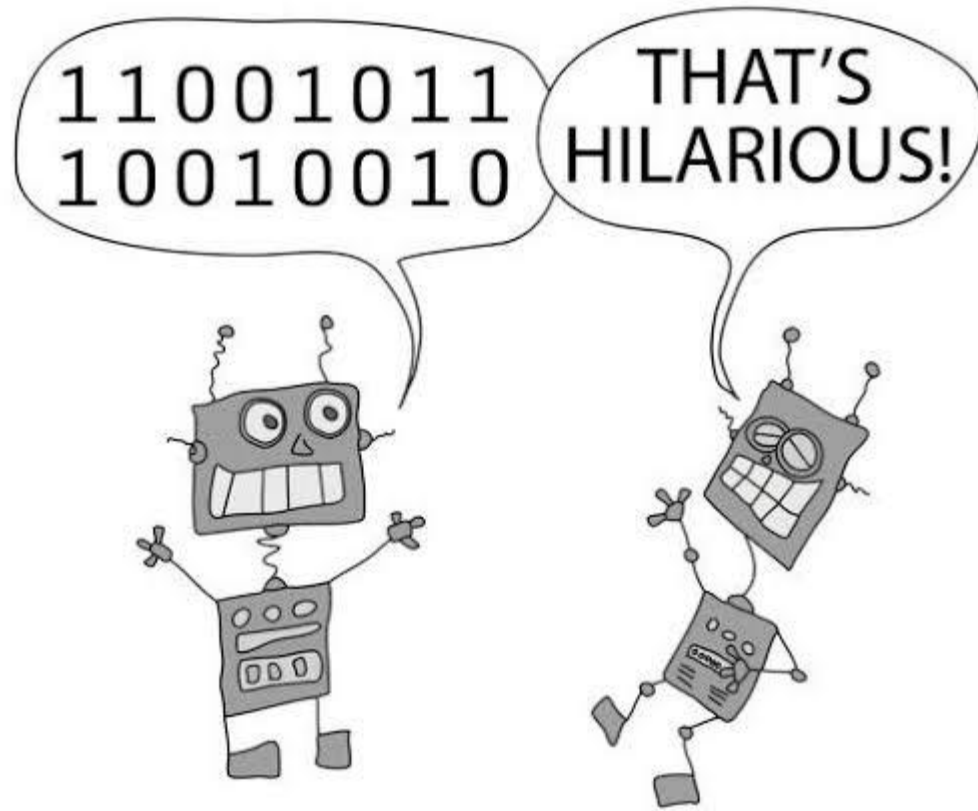
Smartphone



OpenAI ChatGPT

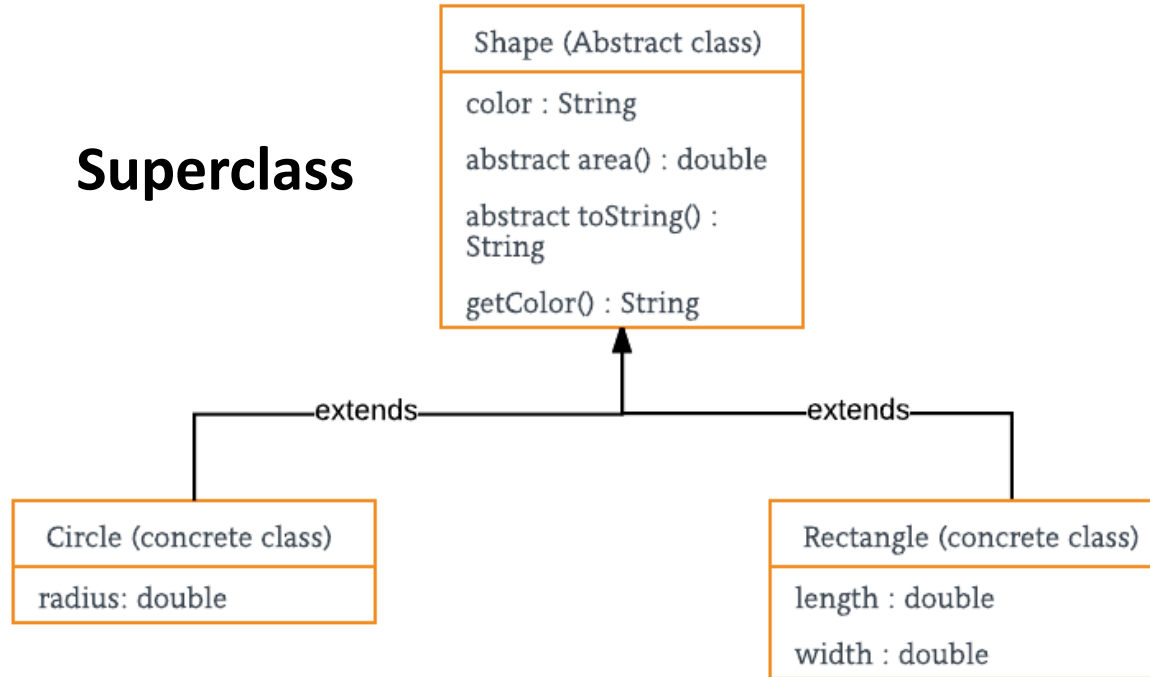


Internet



Semua sudah disiapkan, kita hanya tahu
pakai, tidak tahu untuk menciptakan.
Dipermudah.

Contoh Class Abstract



Ada beberapa situasi di mana kita ingin mendefinisikan **superclass** yang mendeklarasikan **struktur abstraksi** tertentu **tanpa menyediakan implementasi lengkap** dari setiap method. Terkadang kita ingin membuat superclass yang hanya mendefinisikan bentuk generalisasi yang akan digunakan bersama oleh semua subkelasnya, dan menyerahkan kepada setiap subkelas untuk mengisi rinciannya.

Class Abstract Concept



Konsep dari abstraksi hampir sama seperti ketika kita berencana untuk membuat sebuah rumah impian. Dimana, bentuk awal rumah tersebut sudah ada di dalam pikiran kalian, jauh sebelum kalian membuat design rumah kita.

What is abstraction in Python?

- ❖ Abstraction is one of the most important features of object-oriented programming. It is used **to hide the background details or any unnecessary implementation.**
- ❖ Abstraction in python is defined as **hiding the implementation of logic from the client** and using a particular application, and the most important key feature of Object-Oriented Programming. It **hides the irrelevant data specified in the project, reducing complexity and giving value to efficiency.** Abstraction is made in Python using Abstract classes and their methods in the code.
- ❖ Abstraction focuses on **hiding the internal implementations of a process or method from the user.**
- ❖ As the primary role of **Abstraction is to hide the internal functioning of the code**, and the interactions are made with the users through the basic implementation. In other words, the **user knows what he is doing, not how the works are done behind.**

What is abstraction in Python?

Abstraction in python is defined as a process of **handling complexity by hiding unnecessary information from the user**. This is one of the core concepts of object-oriented programming (OOP) languages.

That enables the user to implement even more complex logic on top of the provided abstraction **without understanding or even thinking about all the hidden background/back-end complexity.**



What is “Class Abstract” in Java?

```
package Teori;

public abstract class Parent {
    public abstract void mAbstract();

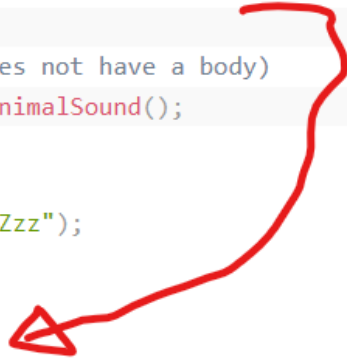
    public static void main(String args[]){
        Parent p = new Parent();
    }
}
```

Tidak bisa membuat
object dari class abstract

- ❖ **Abstract class** adalah class yang mempunyai setidaknya satu **abstract method**.
- ❖ **Abstract method** adalah method yang tidak memiliki *body* (*hanya deklarasi method*).
- ❖ Abstract class tidak bisa dibuat objectnya.
- ❖ Object hanya bisa dibuat dari non-abstract class (*concrete class*).
- ❖ Konsekuensinya, suatu **abstract class** harus diturunkan dimana pada subclass tersebut berisi implementasi dari abstract method yang ada di super class-nya.

Abstract Classes & Methods in Java

An abstract class can have both abstract and regular methods.



```
// Abstract class
1 abstract class Animal {
    // Abstract method (does not have a body)
    public abstract void animalSound();
    // Regular method
    public void sleep() {
        System.out.println("Zzz");
    }
}

2 // Subclass (inherit from Animal)
class Pig extends Animal {
    public void animalSound() {
        // The body of animalSound() is provided here
        System.out.println("The pig says: wee wee");
    }
}

3 class Main {
    public static void main(String[] args) {
        Pig myPig = new Pig(); // Create a Pig object
        myPig.animalSound();
        myPig.sleep();
    }
}
```

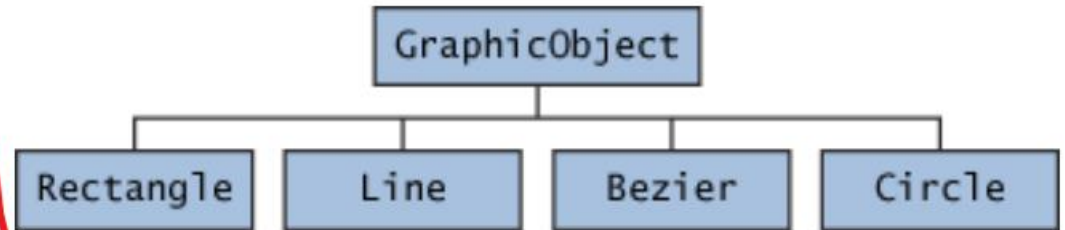
The abstract keyword is a non-access modifier, used for classes and methods:

- 1) **Abstract class:** is a restricted class that cannot be used to create objects (*to access it, it must be inherited from another class*).
- 2) **Abstract method:** can only be used in an abstract class, and it does not have a body. The body is provided by the subclass (*inherited from*).

Contoh Java Class Abstract

```
class Circle extends GraphicObject {  
    void draw() {  
        ...  
    }  
    void resize() {  
        ...  
    }  
}
```

```
class Rectangle extends GraphicObject {  
    void draw() {  
        ...  
    }  
    void resize() {  
        ...  
    }  
}
```

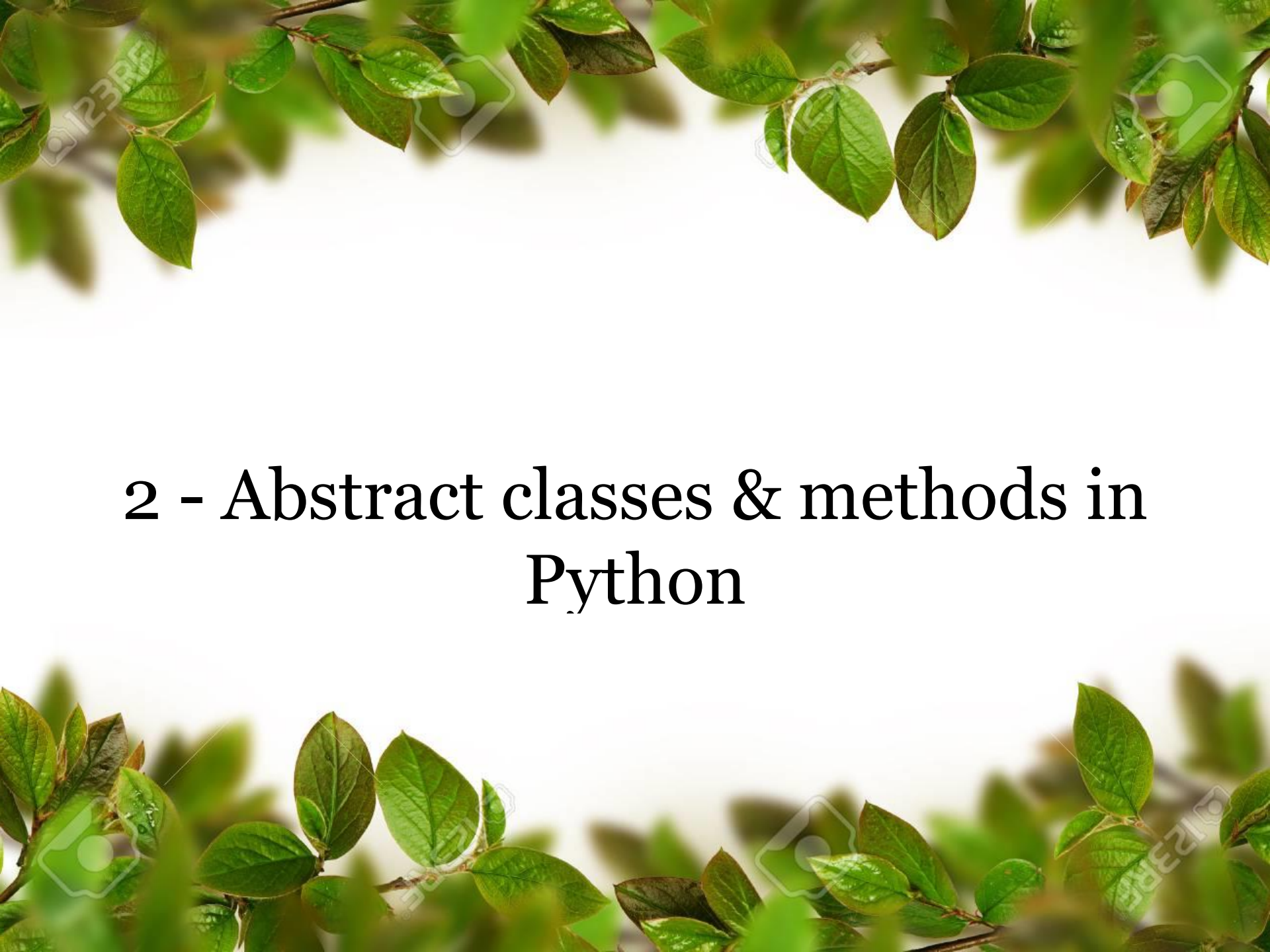


```
abstract class GraphicObject {  
    int x, y;  
    ...  
    void moveTo(int newX, int newY) {  
        ...  
    }  
    abstract void draw();  
    abstract void resize();  
}
```

Why & When To Use Abstract Classes & Methods?



To achieve security –
Hide certain details and only show the important
details of an object.



2 - Abstract classes & methods in Python

Abstract Class & Method in Python

- ❖ In Python, we can achieve abstraction by incorporating **abstract classes** and **abstract methods**. Any class that contains **abstract method(s)** is called an **abstract class**.
- ❖ **Abstract methods do not include any implementations**, it are always defined and implemented as part of the methods of the sub-classes inherited from the abstract class.

Example syntax for an **abstract class**:

```
from abc import ABC
// abc is a library from where a class ABC is being imported. However, a s
eparate class can also be created. The importing from the library has noth
ing to do with abstraction.

Class type_shape(ABC):
```

- 1) The **class type_shape** is inherited from the ABC class (from **ABC module**). In Python, **ABC (Abstract Base Classes)** is a **mechanism that allows us to define abstract classes**.
- 2) An **ABSTRACT CLASS** is a class that cannot be instantiated directly, but is used as a framework to define behavior that must be implemented by its subclasses.

Define An Abstract Method

Define an **abstract method area** inside the **class type_shape**:

```
from abc import ABC
class type_shape(ABC):
    // abstract method area
    def area(self):
        pass
```

```
1 from abc import ABC
2 class product(ABC): # Abstract Class
3
4     # Normal Method
5     def item_list(self, rate):
6         # Method Definition
7         print("amount submitted : ",rate)
8
9     # Abstract Method
10    def product(self,rate):
11        # Method Definition
12        Pass
```

Noted in Python:

- 1) In python, an abstract class can hold both an **abstract method** and a **normal method**.
- 2) The second point is an abstract class is not initiated (**no objects are created**).
- 3) As python doesn't define an **interface**, instead, we can use a Keyword abstract class itself.
- 4) The derived class implementation methods are defined in **abstract base classes**.

```

1 from abc import ABC, abstractmethod
2
3 class Animal(ABC):
4     def __init__(self, name, age, species, color, gender):
5         self.species = species
6         self.color = color
7         self.name = name
8         self.age = age
9         self.gender = gender
10
11     @abstractmethod
12     def get_weight(self):
13         pass
14
15     @abstractmethod
16     def get_location(self):
17         pass
18
19     @abstractmethod
20     def get_health(self):
21         pass
22
23     def get_name(self):
24         return self.name;
25
26     def get_age(self):
27         return self.age;
28
29     def get_gender(self):
30         return self.gender;

```

- 1) Keyword **@abstractmethod** digunakan untuk mendeklarasikan sebuah method sebagai **abstract method** dalam abstract class di Python.
- 2) In abstract methods, using “**keyword pass**” in an abstract method indicates that the method has no concrete implementation (**tidak melakukan apa-apa**) in the class being defined.

Penggunaan Decorator `@abstractmethod`

Di dalam sebuah class abstract, apakah penggunaan decorator "**@abstractmethod**" dapat di hapus?





Exercise *for* Students

Lab Exercise #1

```
DemoAbstract.py X
1  from abc import ABC, abstractmethod
2
3  # class abstract
4  class Animal(ABC):
5      def __init__(self, name, age, species, color, gender):
6          self.species = species
7          self.color = color
8          self.name = name
9          self.age = age
10         self.gender = gender
11
12     @abstractmethod
13     def get_weight(self):
14         pass
15
16     @abstractmethod
17     def get_location(self):
18         pass
19
20     @abstractmethod
21     def get_health(self):
22         pass
23
24     def get_name(self):
25         return self.name;
26
27     def get_age(self):
28         return self.age;
29
30     def get_gender(self):
31         return self.gender;
32
33     def show_informations(self):
34         print(self.get_name());
35         print(self.get_age());
36         print(self.get_weight());
37         print(self.get_location());
38         print(self.get_gender());
39         print(self.get_health());
40
```

```
2  class Mamalia(Animal):
3      def get_weight(self):
4          return "1 Kilo"
5
6      def get_location(self):
7          return "Bitung"
8
9      def get_health(self):
10         return "Sick"
11
```

```
3  class Bird(Animal):
4      def get_weight(self):
5          return "0.5 Kilo"
6
7      def get_location(self):
8          return "Tropical forest"
9
10     def get_health(self):
11         return "Healthy"
12
```

```
4  class Fish(Animal):
13     def get_weight(self):
14         return "2 Kilos"
15
16     def get_location(self):
17         return "Ocean"
18
19     def get_health(self):
20         return "Moderate"
21
```

```
71 # objects are created and the parameterized constructor is called
72 obj_cat = Mamalia('Kucing', '20 Tahun', 'Mamalia', 'white', 'Female') # object 1
73 obj_bird = Bird('Elang', '15 Tahun', 'Bird', 'white', 'Female') # object 2
74 obj_salmon = Fish('Salmon', '1 Tahun', 'Fish', 'Blue', 'Female') # object 3
75
76 # user-defined function is called from object 1
77 obj_cat.show_informations();
78 print("");
79 obj_bird.show_informations();
80 print("");
81 obj_salmon.show_informations();
```

class Bird(Animal):
Ini berarti **Bird** adalah sebuah kelas yang diturunkan dari (**mewarisi**) kelas Animal. Class Bird memiliki semua metode dan atribut yang didefinisikan dalam kelas Animal, dan dapat menambahkan metode dan atribut tambahan jika diperlukan.

In Python, **class Bird(Animal)**, **class Fish(Animal)**, and **class Mammal(Animal)** are examples of what is called "**inheritance**". In this context, these classes are referred to as "subclasses", while class Animal is referred to as the "parent class".

```
[6]: runfile('D:/UNKLAB CLASSES/CLASS/Object-Oriented Programming/  
CODES/Lecture8 [abstract]/DemoAbstract.py', wdir='D:/UNKLAB CLASSES/  
CLASS/Object-Oriented Programming/CODES/Lecture8 [abstract]')  
  
Kucing  
20 Tahun  
1 Kilo  
Bitung  
Female  
Sick  
  
Elang  
15 Tahun  
0.5 Kilo  
Tropical forest  
Female  
Healthy  
  
Salmon  
1 Tahun  
2 Kilos  
Ocean  
Female  
Moderate
```

Modifikasi source code pada file DemoAbstract.py:

1. Dalam **class Animal**, students diminta untuk mengubah method ***get_name()*** dan method ***get_age()*** menjadi dua buah *abstract method*. Dimana masing-masing method akan memiliki **parameter input** yaitu variable ***nama*** untuk method ***get_name()*** dan variable ***umur*** untuk ***get_age()***. Method-method tersebut akan melakukan **inisialisasi class variable** dan tanpa mengembalikan nilai dari *class variables* yang terkait.
2. Method ***get_name()*** dan method ***get_age()*** akan di panggil dari luar class menggunakan ***object*** dari masing-masing class yang telah dibuat.
3. Buat sebuah class baru dengan nama **class Amfibi**. Dimana class ini yang diturunkan dari (*mewarisi*) kelas Animal. Class Amfibi ini akan memiliki method sama seperti class Mamalia, class Bird dan/atau class Fish.
4. Buat **object** dari class **Amfibi** dan tampilkan informasi karakteristik dengan menggunakan method ***show_information()***.

```

36     def show_informations(self):
37         print(self.name);
38         print(self.age);
39         print(self.get_weight());
40         print(self.get_location());
41         print(self.get_gender());
42         print(self.get_health());
43
44     class Mamalia(Animal):
45         def get_name(self, nama):
46             self.name = nama;
47
48         def get_age(self, umur):
49             self.age = umur;
50
51         def get_weight(self):
52             return "1 Kilo"
53
54         def get_location(self):
55             return "Bitung"
56
57         def get_health(self):
58             return "Sick"
59
60     class Bird(Animal):
61         def get_name(self, nama):
62             self.name = nama;
63
64         def get_age(self, umur):
65             self.age = umur;
66
67         def get_weight(self):
68             return "0.5 Kilo"

```

```

91 # objects are created and the parameterized constructor is called
92 obj_cat = Mamalia('Kucing', '20 Tahun', 'Mamalia', 'white', 'Female') # object 1
93 obj_bird = Bird('Elang', '15 Tahun', 'Bird', 'white', 'Female') # object 2
94 obj_salmon = Fish('Salmon', '1 Tahun', 'Fish', 'Blue', 'Female') # object 3
95
96 obj_cat.get_name("Kucing Hitam")
97 obj_cat.get_age("50 Tahun")
98 obj_bird.get_name("Burung Gereja")
99 obj_bird.get_age("30 Tahun")
100 obj_salmon.get_name("Lumba-Lumba")
101 obj_salmon.get_age("10 Tahun")
102

```

Output Program

```

Kucing Hitam
50 Tahun
1 Kilo
Bitung
Female
Sick

Burung Gereja
30 Tahun
0.5 Kilo
Tropical forest
Female
Healthy

Lumba-lumba
10 Tahun
2 Kilos
Ocean
Female
Moderate

```


END PRESENTATION

Thank you for your attention

Instructor: S – W – T

THANK YOU

